mathematics

MDPI

Article

An Importance Sampling Method for Generating Optimal Interpolation Points in Training Physics-Informed Neural Networks

Hui Li [1,*], Yichi Zhang [1], Zhaoxiong Wu [2], Zhe Wang [2] and Tong Wu [2]

[1] Department of Computer Science and Technology, Beijing University of Chemical Technology, Beijing 100029, China; 2023200835@buct.edu.cn
[2] Beijing Institute of Computer Technology and Applications, Beijing 100854, China; yangguangxiaoxiong@126.com (Z.W.); wangxiaozhe1026@163.com (Z.W.); wutongegw@163.com (T.W.)
* Correspondence: ray@mail.buct.edu.cn

Abstract: The application of machine learning and artificial intelligence to solve scientific challenges has significantly increased in recent years. A remarkable development is the use of Physics-Informed Neural Networks (PINNs) to solve Partial Differential Equations (PDEs) numerically. However, current PINN techniques often face problems with accuracy and slow convergence. To address these problems, we propose an importance sampling method to generate optimal interpolation points during training. Experimental results demonstrate that our method achieves a 43% reduction in root mean square error compared to state-of-the-art methods when applied to the one-dimensional Korteweg–De Vries equation.

Keywords: physics-informed neural networks; importance sampling; partial differential equations; discrete wavelet transform

MSC: 35A02

check for updates

Academic Editors: Huaizhong Zhao and Ke-Lin Du

Received: 14 November 2024
Revised: 28 December 2024
Accepted: 1 January 2025
Published: 3 January 2025

Citation: Li, H.; Zhang, Y.; Wu, Z.; Wang, Z.; Wu, T. An Importance Sampling Method for Generating Optimal Interpolation Points in Training Physics-Informed Neural Networks. *Mathematics* **2025**, *13*, 150. https://doi.org/10.3390/math13010150

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Partial Differential Equations (PDEs) depict physical phenomena and are crucial in numerous scientific and engineering fields [1,2]. Examples include modeling airflow around airfoils using Navier-Stokes equations [3], optical design with Maxwell's equations [4], and heat transfer equations in metallic or plastic parts manufacturing [5]. However, their significant nonlinearity makes them challenging to solve. Conventional numerical approaches employ discretization techniques to approximate solutions [6], such as the Finite Element Method (FEM) [7] and the Finite Difference Method (FDM) [8]. Nevertheless, these methods require substantial computational resources and often need careful task-specific customization to ensure numerical stability. Furthermore, when dealing with high-dimensional PDEs, the exponential growth in grid points with increasing dimensions leads to the "curse of dimensionality", resulting in prohibitive memory and computational time requirements that may be impractical for real-world applications.

Recent advances in artificial intelligence have spurred its use in solving scientific problems [9–11], including neural networks-based solutions for PDEs. Various neural network frameworks have been proposed, such as Physics-Guided Neural Networks (PGNN) [12], Physics-Informed Neural Networks (PINN) [13], Physics-Encoded Neural Networks (PENN) [14], and Neural Operators (NO) [15]. Among them, PINN has received attention for its unique features [16]:

- Mesh-free: PINNs eliminate the need for discrete mesh or grid discretization, instead building models directly on continuous domains. By leveraging neural networks' approximation capabilities to model physical field distributions, this mesh-free approach offers greater flexibility in handling complex geometries and high-dimensional problems.
- Inverse Problems: These involve inferring unknown PDE parameters from observed data. PINNs elegantly solve inverse problems by treating unknown parameters as trainable neural network parameters.
- Data Efficiency: PINNs leverage physics-based prior knowledge, enabling model training with sparse labeled or unlabeled data.
- Unified Framework: PINNs require no special design modifications for different specific tasks.

PINNs incorporate the constraints of the differential equation into the loss function of the neural network. The minimization of this function allows the neural network to adhere to the PDE. For example, in the context of a heat transfer equation, PINNs integrate the network's output directly into the heat conduction equation to calculate the "residual" in the loss function. This approach ensures that the solution adheres closely to the governing physical laws. Additionally, the initial temperature distribution and boundary conditions of the problem are explicitly incorporated into the loss function, allowing the network to simultaneously satisfy both the initial conditions and boundary constraints throughout the solution process.

Essentially, PINNs transform the process of solving a differential equation into an optimization problem within the neural network. This method enables the selection of points within the time and space domain of the equation for training, thereby eliminating the need for a fixed grid or labeled data.

Nevertheless, PINNs require more computational power and may converge at a slower rate during training compared to other methods. Consequently, research on PINNs is primarily focused on improving their convergence speed and accuracy. Key strategies include refining the network structure [17–19], optimizing the selection of training samples [20,21], dividing regions for parallel computing [22], and modifying the distribution of loss function weights [23]. PINNs' loss function comprises multiple terms, and the methods modifying the distribution of loss function weights primarily aim to balance these terms. Most methods require customization for specific physical systems. While parallel computing methods achieve PINN parallelization by decomposing the solution domain into subdomains, this decomposition requires additional boundary conditions at subdomain interfaces, potentially affecting model accuracy. Of these strategies, refining the sampling strategy is a flexible approach as it does not require specific customizations for various differential equations and can integrate easily with other improvement methods. Daw et al. [24] investigated the reasons for the failure of PINN propagation, primarily focusing on enhancing the sampling strategy. They emphasized the significant influence of the sampling strategy on PINN convergence and suggested a dynamic residual-based sampling scheme. Similarly, Wu et al. [25] compared multiple uniform and adaptive sampling schemes, both non-adaptive and adaptive, establishing that suitable adaptive schemes can improve the PINN accuracy. Mao and Meng [26] proposed a gradient and residual-based adaptive sampling scheme that improved PINN accuracy for PDEs with steep solutions.

Nabian et al. [20] applied importance sampling to the PINN sampling process. However, the method of [20] generates and fixes interpolation seeds uniformly, lacking the ability to dynamically introduce them into challenging regions during model training. Yang et al. [21] improved importance sampling weight estimation accuracy through Dynamic Mesh-Based Importance Sampling (DMIS). Although Yang et al.'s approach allows

for better interpolation seeds in these areas, the seeds are limited to selection from collection points. This makes accurate estimation challenging in regions with significant changes.

In this paper, we propose an improved sampling method that incorporates wavelet transforms into the selection of dynamic interpolation points for PINN training's importance sampling. Specifically, we use wavelet transforms to identify the most dramatically changing regions during PINN training and increase interpolation points in these regions to improve the accuracy of importance sampling weight estimation.

The main contributions of this paper are as follows:

- We proposed an improved method for estimating weight in importance sampling to improve model accuracy and the convergence rate of the PINN.
- We tested the proposed method using two widely accepted equations as benchmarks.
- We demonstrated the correlation between areas with a significant change in predicted value and areas with a significant change in loss experimentally within a PINN.

The rest of this paper is structured as follows: Section 2 presents fundamental techniques such as the physics-informed neural network, importance sampling, and discrete wavelet transform, serving as a foundation for Section 3. Section 3 outlines methods for extracting significantly changing areas and selecting optimal interpolation points. Section 4 tests the proposed methods and provides some discussion. Finally, Section 5 concludes the paper.

2. Preliminaries

2.1. Physics Informed Neural Network

The Physics Informed Neural Network (PINN) [13] embeds partial differential equations (PDEs) into the loss function of the neural network, establishing a neural network model $f_\theta(x,t)$ to provide a numerical solution for differential equations, where θ is the parameter of the neural network. Consider the general form of a nonlinear partial differential equation:

$$\begin{cases} u_t + \mathcal{N}(u) = 0, & x \in \Omega, t \in [0,T] \\ u(x,0) = h(x), & x \in \Omega \\ u(x,t) = g(x,t), & x \in \partial\Omega \end{cases} \tag{1}$$

where $u(x,t)$ represents the solution of the partial differential equation, $h(x)$ the initial condition of the equation, $g(x,t)$ represent the boundary condition of the equation, and $\mathcal{N}(\cdot)$ is a general differential operator. Ω is the spatial domain, $\partial\Omega$ represents the boundary of the spatial domain, and T is the time range.

For non-initial points and non-boundary points ($x \notin \partial\Omega$ and $t \neq 0$), the residual is represented as follows:

$$R_r(x,t;\theta) = \frac{\partial}{\partial t} f_\theta(x,t) + \mathcal{N}(f_\theta(x,t)). \tag{2}$$

For initial points ($t = 0$), the residual is represented as follows:

$$R_{init}(x,0;\theta) = f_\theta(x,0) - h(x). \tag{3}$$

For boundary points ($x \in \partial\Omega$), the residual is represented as follows:

$$R_b(x,t;\theta) = f_\theta(x,t) - g(x,t). \tag{4}$$

The loss function of the model can be defined as the sum of squares of the residuals at all points in the time and space domain:

$$\begin{cases} L_r(\theta) = \int_{[0,T]\times\Omega} [R_r(x,t;\theta)]^2 dxdt \\ L_{init}(\theta) = \int_{\Omega} [R_{init}(x,0;\theta)]^2 dx \\ L_b(\theta) = \int_{[0,T]\times\partial\Omega} [R_b(x,t;\theta)]^2 dxdt \\ L(\theta) = \lambda_r L_r(\theta) + \lambda_{init} L_{init}(\theta) + \lambda_b L_b(\theta) \end{cases} \tag{5}$$

where $\lambda_r, \lambda_{init}, \lambda_b$ are the weights of losses. Figure 1 presents a flowchart illustrating how the residuals from the initial, boundary, and internal points combine to form the loss function.

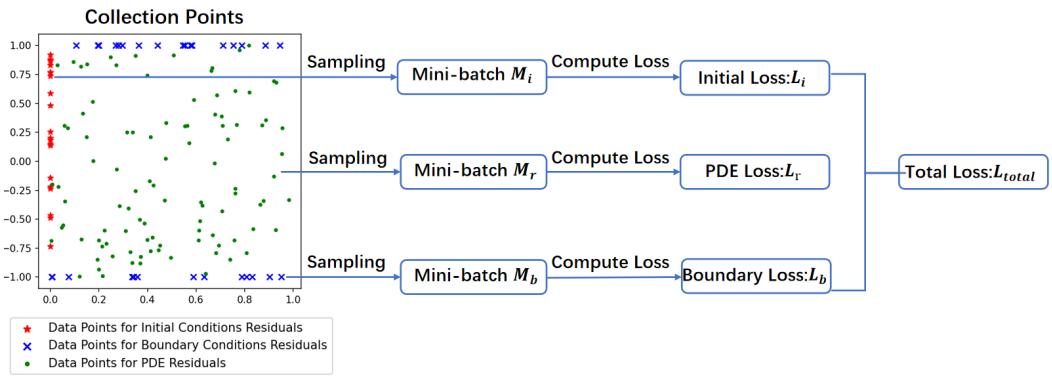

Figure 1. A flowchart showing how the loss from the initial, boundary, and internal points combine to form the loss function.

A schematic diagram of a PINN is shown in Figure 2. The parameter optimization problem in the neural network can be defined as the following unconstrained optimization problem:

$$\theta^* = \arg\min_{\theta}(L(\theta)). \tag{6}$$

Since the continuous integrals in Equation (5) cannot be calculated directly, typically N points are chosen by uniformly sampling in the input area $[0,T] \times \Omega$. The unconstrained optimization problem can be addressed using the mini-batch Stochastic Gradient Descent (SGD) algorithm [27]. Each iteration selects m points from all collocation points as a mini-batch, and the parameters of the neural network θ are updated through backpropagation [27] to approximate the optimal parameters θ^*. The loss function of the mini-batch in the k-th iteration is formulated as follows:

$$L(\theta^{(k)}) = \frac{1}{m} \sum_{j \in M^{(k)}} [R_r(x_j, t_j; \theta^{(k)}) + R_{init}(x_j, 0; \theta^{(k)}) + R_b(x_j, t_j; \theta^{(k)})], \tag{7}$$

where $M^{(k)}$ represents the index set of samples in the mini-batch selected in the k-th iteration. The model parameters are updated as follows:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_\theta L(\theta^{(k)}), \tag{8}$$

where $\nabla_\theta L$ is obtained by backpropagation, and η is the learning rate. The optimization process of minimizing the loss function faces several challenges: The PDE-constrained loss function typically involves high-order derivatives of neural network outputs with respect to input variables, potentially leading to vanishing or exploding gradients, especially in deep architectures or when loss terms have significant scale differences. Additionally, the physics-informed loss landscape is highly non-convex, increasing the risk of convergence to

local minima or saddle points. These challenges are exacerbated when the problem involves complex or stiff differential equations. Furthermore, computing high-order derivatives in PINN's loss function requires repeated gradient calculations of network parameters and input variables, making it computationally intensive.

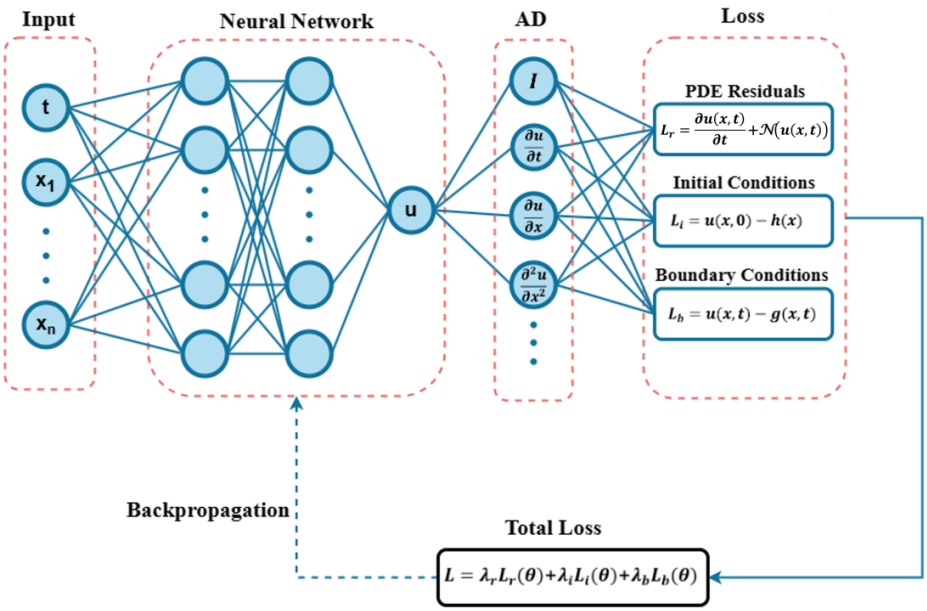

Figure 2. Schematic diagram of PINN structure. The inputs are space and time coordinates (x, t). A fully connected neural network is used to approximate the solution to the PDE. The derivative of the output u corresponding to the input is calculated using Automatic Differentiation (AD) [28]. The loss function, which facilitates backpropagation to learn the parameters of the neural network, is the weighted sum of several residuals.

For example, consider the heat equation mentioned in the introduction:

$$\begin{cases} \dfrac{\partial u}{\partial t} - \alpha \dfrac{\partial^2 u}{\partial x^2} = 0, \quad x \in [0, H],\ t > 0, \\ u(x, 0) = u_0(x) \\ u(0, t) = u_L, u(H, t) = u_R. \end{cases} \tag{9}$$

The loss of PINN is as follows:

$$\begin{cases} L_{\mathrm{r}} = \dfrac{1}{N_r} \displaystyle\sum_{i=1}^{N_r} \left| \dfrac{\partial u}{\partial t}(x_i, t_i) - \alpha \dfrac{\partial^2 u}{\partial x^2}(x_i, t_i) \right|^2 \\ L_{\mathrm{b}} = \dfrac{1}{N_b} \displaystyle\sum_{i=1}^{N_b} \left(|u(0, t_i) - u_L|^2 + |u(L, t_i) - u_R|^2 \right) \\ L_{\mathrm{init}} = \dfrac{1}{N_{init}} \displaystyle\sum_{i=1}^{N_{init}} |u(x_i, 0) - u_0(x_i)|^2 \\ L_{\mathrm{total}} = L_{\mathrm{r}} + L_{\mathrm{b}} + L_{\mathrm{init}}. \end{cases} \tag{10}$$

We use randomly sampled N_r collocation points (x_i, t_i) in the domain for L_{r}, N_b points on the boundaries $(0, t_i)$ and $(1, t_i)$ for L_{b}, and N_{init} points at $t = 0$ for L_{init}, then minimize L_{total} using gradient-based optimization.

Due to this modeling approach, PINNs can be trained without labeled data, instead using the embedded PDE equations as "supervisors". Unlike numerical methods such as finite elements or finite differences, PINNs do not require domain discretization into meshes,

avoiding computational intensity in high-dimensional or complex domains. Furthermore, PINNs eliminate the need for the special handling of initial and boundary conditions typical in traditional numerical methods, offering a unified framework that integrates these constraints into the loss function without additional special treatment.

2.2. Importance Sampling

Importance sampling [29] is a useful method for computing Monte Carlo integrals [30], as it can decrease the variance of the estimate by using a sampling distribution. The concept behind it is that certain values of a random variable are more significant on the estimate than others. By sampling these impactful values more frequently, the estimate's variance can be reduced. To illustrate this concept, consider a survey investigating product popularity across different age groups. If we know a priori that the product primarily appeals to young people, it makes sense to survey more individuals from this demographic to better understand product performance. By focusing efforts on this age group, we can reduce the uncertainty (or variance) in our popularity estimates more effectively than by surveying all age groups equally. Similarly, in importance sampling, we bias sampling toward the most significant regions of the target function, ensuring more precise estimates with fewer samples. However, using the adjusted sampling distribution directly could lead to a biased estimate. Therefore, it is crucial to apply weights to the samples to preserve the unbiasedness of the estimate [31].

Consider a random variable X with a probability density of $p(x)$. For a function $f(X)$ of X, if we want to calculate the expected value $\mu_f = \mathbb{E}_p[f(X)]$, we calculate

$$\mu_f = \int f(x)p(x)dx. \tag{11}$$

Then, for any probability density function $q(x)$ that satisfies $q(x) > 0$ when $f(x)p(x) \neq 0$, we have the following:

$$\mu_f = \mathbb{E}_q[w(X)f(X)], \tag{12}$$

where $w(x) = \frac{p(x)}{q(x)}$, and $\mathbb{E}_q[\cdot]$ is the expectation with respect to the distribution $q(x)$. We can estimate μ_f by sampling $x_1, x_2, \cdots, x_m$ with $q(x)$:

$$\widehat{\mu}_f = \frac{1}{m}\sum_{i=1}^{m} w(x_i)f(x_i). \tag{13}$$

The design of the sampling distribution is crucial for implementing importance sampling [32]. The distribution used for PINN training in importance sampling is given in the following subsection.

2.3. Importance Sampling for PINN

Uniform sampling is commonly used in the creation of mini-batches [33]. Given that the computation of loss in the mini-batch is fundamentally a Monte Carlo integration process, a more efficient method, namely importance sampling [34], can be applied to estimate $L(\theta^{(k)})$:

$$\widehat{L}(\theta^{(k)}) \approx \frac{1}{m}\sum_{i=1}^{m} \frac{p_i}{q_i} R_r(x_i, t_i; \theta^{(k)}), \tag{14}$$

where p represents the probability distribution of the sampling points in the input domain $[0, T] \times \Omega$, and q represents the alternative sampling distribution implemented in the importance sampling method.

The continuous domain $[0, T] \times \Omega$ is transformed into N discrete sampling points through discrete sampling. This allows us to convert the continuous distribution function

into discrete distributions p_i for $i = 1, 2, \cdots, N$ and q_i for $i = 1, 2, \cdots, N$. Here, $p_i = \frac{1}{N}$ because the sampling points come from uniform sampling. The model parameter is updated as follows:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_\theta \left(\frac{1}{m} \sum_{i=1}^{m} \frac{1}{N q_i} R_r(x_i, t_i; \theta^{(k)}) \right). \tag{15}$$

The convergence speed can be defined as follows:

$$S^{(k)} = -\mathbb{E}(\|\theta^{(k+1)} - \theta^*\|_2^2 - \|\theta^{(k)} - \theta^*\|_2^2), \tag{16}$$

where $S^{(k)}$ is the convergence speed at the k-th iteration.

Zhao and Zhang [35] and Nabian et al. [20] have demonstrated that when the sampling probability is proportional to the 2-norm of the loss gradient of all parameters of a sample, i.e., $q \propto \|\nabla_\theta R_r(x, t; \theta)\|_2$, the highest convergence speed can be achieved. In this case,

$$q_i = \frac{\|\nabla_\theta R_r(x_i, t_i; \theta^{(k)}\|_2}{\sum_{i=1}^{N} \|\nabla_\theta R_r(x_i, t_i; \theta^{(k)})\|_2}. \tag{17}$$

While theoretically optimal, calculating the 2-norm of gradients for all collocation points in each iteration requires additional backpropagation through the computational graph, which is computationally prohibitive. Katharopoulos and Fleuret [36] have both theoretically and experimentally shown that the linear transformation of the loss is consistently larger than the 2-norm of the gradient during training. They also found that the ordering of collocation points by their 2-norm is almost the same as the ordering by their transformed losses. Therefore, the transformed loss can be used as an approximation of the 2-norm of the loss, serving as a measure for importance sampling. Hence, an approximation for q_i can be obtained as follows:

$$q_i \approx \frac{R_r(x_i, t_i; \theta^{(k)})}{\sum_{i=1}^{N} R_r(x_i, t_i; \theta^{(k)})}. \tag{18}$$

2.4. Discrete Wavelet Transform

The wavelet transform [37] uses finite-length or quick-decaying oscillatory waveforms, known as the "mother wavelet", to represent a signal. A specific type of wavelet transform is the Discrete Wavelet Transform (DWT) [38], which discretely samples the wavelet. DWT is commonly used in dealing with two-dimensional discrete signals [39–41]. DWT can capture both frequency and position information which is particularly useful when the frequency distribution of a signal significantly varies with position [42].

For 2D discrete signals, the DWT process with Haar wavelet basis can be described as follows. The Haar scale function is as follows:

$$\varphi(x) = \begin{cases} 1, & 0 \le x \le 1 \\ 0, & \text{otherwise} \end{cases} \tag{19}$$

and the Haar wavelet function is as follows:

$$\psi(x) = \begin{cases} 1, & 0 \le x < 0.5 \\ -1, & 0.5 \le x < 1 \\ 0, & \text{otherwise} \end{cases} \tag{20}$$

Hence, the 2D scale function and 2D wavelet functions are as follows:

$$\begin{cases} \varphi(x,y) = \varphi(x)\varphi(y) \\ \psi^H(x,y) = \psi(x)\varphi(y) \\ \psi^V(x,y) = \varphi(x)\psi(y) \\ \psi^D(x,y) = \psi(x)\psi(y). \end{cases} \tag{21}$$

The 2D scale and translation basis functions:

$$\begin{cases} \varphi_{m,n}(x,y) = \sqrt{2}\varphi(2x-m, 2-n) \\ \psi^h_{m,n}(x,y) = \sqrt{2}\psi(2x-m, 2-n), h \in \{V, H, D\}. \end{cases} \tag{22}$$

Therefore, the 2D DWT can be formulated as follows:

$$\begin{cases} W_\varphi(m,n) = \dfrac{1}{\sqrt{ML}} \displaystyle\sum_{x=0}^{M-1} \sum_{y=0}^{L-1} f(x,y)\varphi_{m,n}(x,y) \\ W^h_\psi(m,n) = \dfrac{1}{\sqrt{ML}} \displaystyle\sum_{x=0}^{M-1} \sum_{y=0}^{L-1} f(x,y)\psi^h_{m,n}(x,y), h \in \{H, V, D\}. \end{cases} \tag{23}$$

where $f(x,y)$ is the original two-dimensional discrete signal, with M and L being its horizontal and vertical lengths. $W_\varphi(m,n)$ denotes the low-frequency subband in wavelet decomposition. $W^h_\psi(m,n)$, where $h \in \{H, V, D\}$, corresponds to the horizontal, vertical, and diagonal high-frequency subbands, respectively.

In PINN training, DWT can be applied to data processing during sampling. Specifically, DWT helps identify regions with high gradients or discontinuities, allowing for increased training points in these areas to improve accuracy. For example, consider a sinusoidal wave with occasional sharp peaks, representing a signal with both low-frequency trends and high-frequency spikes. Applying DWT decomposes the signal into approximation coefficients capturing smooth low-frequency components and detail coefficients capturing sharp high-frequency changes. Analyzing these coefficients helps identify regions with significant features (e.g., peak), providing a foundation for sampling in the PINN training process.

3. Method

Despite using Equation (18) for approximate calculations, significant computation is still required to determine the losses for all collocation points. It is not feasible to accurately calculate the loss for every collocation point during each mini-batch training session. Therefore, existing methods like [20,21], which optimize the PINN sampling process based on importance sampling, use interpolation techniques. These techniques involve accurately calculating the losses for a limited number of points (seed points) at a time. The losses for the remaining points are then estimated using various interpolation methods. For example, in the solution domain, we generate N collocation points and sample $0.1N$ points as seed points. After accurately calculating the loss values at these seed points, we determine the loss values for the remaining collocation points using interpolation methods based on nearby sampling points, such as nearest-neighbor interpolation (where an interpolation point's loss equals that of its nearest seed point).

However, the loss distribution for the neural network model during training is not uniform. In areas with dramatic loss changes, the estimated interpolation value often significantly deviates from the actual value. In small regions where loss values vary significantly between adjacent points, static sampling methods for generating interpolation

seed points often fail to accurately estimate loss values. One possible solution is to increase the number of accurately calculated seed points in areas with high loss fluctuations.

It has been observed that after a certain amount of PINN training, the location of the sampling points with large loss changes strongly correlates with areas where the model's predicted value changes dramatically. Therefore, we propose a sampling method that uses wavelet transform to identify points where the model's predicted value changes significantly during training. This strategy can increase the number of accurately calculated points in areas with significant loss changes, thereby improving the interpolation accuracy in these areas.

3.1. Extracting Dramatically Changing Areas Using Discrete Wavelet Transform

Inspired by the use of DWT in image processing, we employ this method to filter high frequencies from discrete sampling points in the input domain of the PINN. The approach assists in identifying points that exhibit more dramatic changes compared to their surroundings. Specifically, we conduct uniform sampling in the input domain $\Omega \times [0, T]$. Each dimension is uniformly divided into N segments, which produces the sampling point set $\{(x_i, t_i)\}_{i=1}^{N^d}$, where d represents the number of dimensions in the input domain, and N^d is the d-th power of N and represents the total number of sampling points.

For each sampling point, we calculate the corresponding model output $y_i = f_\theta(x_i, t_i)$. $\{y_i\}_{i=1}^{N^d}$ is then used as the input signal for the wavelet transformation with a Haar wavelet base. After removing low-frequency components, we perform the wavelet inverse transformation to obtain the high-frequency part $\{y_i'\}_{i=1}^{N^d}$.

Fitting PDEs with PINNs can be challenging due to their inherent nonlinearity, which often results in difficult-to-train areas marked by unstable losses during each iteration. Interestingly, these areas may correspond to significant changes in the $u(x, t)$ value. After training the PINN model for a certain period, the model's predicted value starts to approximate the actual $u(x, t)$ value. This makes it possible to identify parts of the $u(x, t)$ value that change significantly based on the model's predicted value and to identify areas where the loss fluctuates. Using DWT to extract regions of dramatic change effectively identifies areas where seed point sampling density should be increased, thus providing guidance for more efficient sampling. Figure 3 demonstrates the process of extracting the dramatically changing area in the Burgers equation using the DWT.

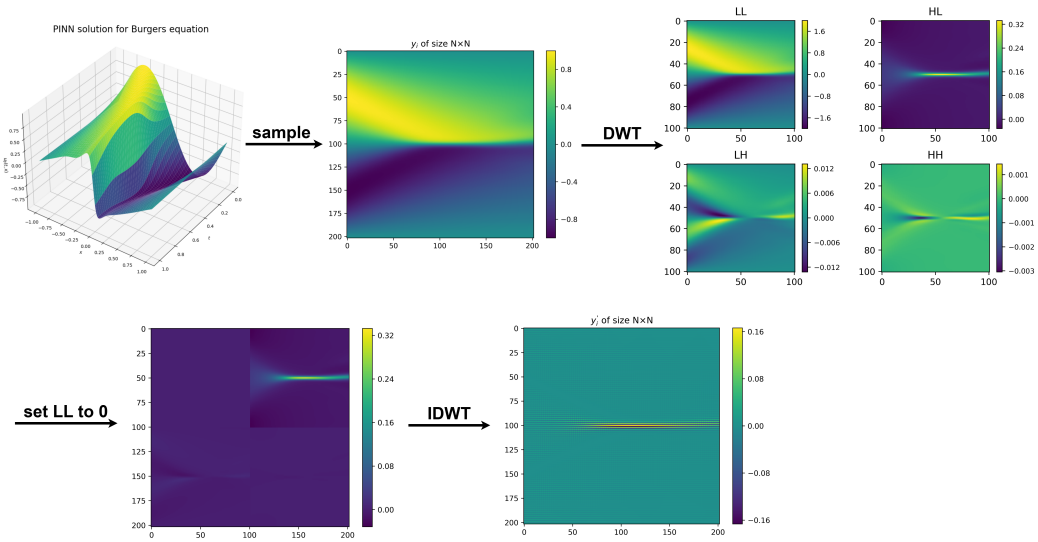

Figure 3. The process of extracting the dramatically changing area of the Burgers equation using DWT.

3.2. Selecting Optimal Interpolation Points

The approximation in Equation (18) streamlines the calculation of the sampling probability in importance sampling. However, the precise calculation of losses for all collection points in each training iteration is still computationally demanding. To mitigate this, we employ an interpolation method, which precisely calculates losses for select seed points. We then estimate the losses of nearby points based on these seed points to compute their sampling probabilities.

The accuracy of this spatial interpolation is significantly influenced by spatial variability. Rapid and inconsistent changes within a local range can heavily impact the effectiveness of the interpolation. While increasing the density of interpolation seed points can enhance precision, it also escalates the number of values requiring precise calculation, thereby increasing computational costs.

From previous observations, we can discern areas with significant model loss changes by identifying regions with significant changes in model-predicted values. It is an effective strategy to increase interpolation seed points in these areas and decrease them in smoother regions. This method achieves superior interpolation results without escalating the number of seed points. We refer to these strategically placed seed points in areas with significant changes as Optimal Interpolation Points (OIP) in this paper.

Given that the distribution of the high-frequency feature matrix is challenging to use directly as a sampling probability, as its dense value distribution would cause sampling to over-concentrate in extremely small regions, distribution equalization becomes necessary. We employ a histogram equalization method [43], commonly used in image processing, for distribution equalization. The distributions before and after equalization are compared in Figure 4.

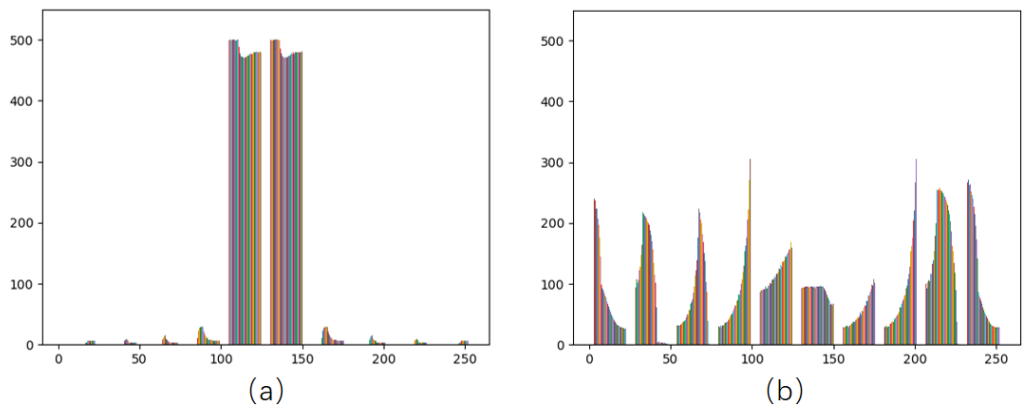

Figure 4. Comparison of distribution histogram before and after equalization. (**a**) histogram of the distribution before equalization (**b**) histogram of the distribution after equalization.

Specifically, we linearly map the high-frequency portion, y_i', with a size of $N \times N$ in the predicted value of the model (see Section 2.1) to an integer between 0 and 255, denoted as y_i''.

Next, we use the cumulative distribution function to equalize the distribution:

$$S(k) = \sum_{j=0}^{k} \frac{n_j}{n}, \quad k = 0, 1, 2, \cdots, 255, \tag{24}$$

where n is the total number of values, here it is equal to N^2, and n_j is the number of values which is equal to j. After mapping $s_i = S(y_i'')$, we obtain the matrix s_i that represents the equalized high-frequency part.

To control the sampling concentration in the area with significant loss changes, we introduce a parameter β to adjust the sampling probability of seed points as follows:

$$p_i' = \frac{s_i^\beta}{\sum_{i=1}^{N} s_i^\beta}, \quad \beta \in [0, \infty), \tag{25}$$

where s_i represents the feature matrix corresponding to the i-th sampling point. As the value of β increases, sampling becomes more concentrated in areas with dramatic changes. As the β value decreases, sampling becomes more uniformly distributed across the entire space. According to Equation (25), when β equals zero, sampling degenerates to uniform sampling. For model training, excessively high β values cause interpolation seeds to be too concentrated, leading to inaccurate loss estimates in regions without dramatic changes, affecting the overall model performance. Conversely, overly low β values result in near-uniform sampling, losing the ability to dynamically increase sampling points in critical regions.

Generally, areas of dramatic change tend to stabilize after a certain stage of model training. Therefore, we use a smaller β during the early training stages and gradually increase β as training progresses, incrementally raising the number of interpolation seed points in areas of dramatic change. However, to ensure a sufficient number of interpolation seed points in other areas, only 50% of the seed points are generated using the optimal interpolation point method. The rest are generated through uniform sampling. The process for selecting optimal interpolation points based on changes in model predicted values is outlined in Algorithm 1, while the method for training PINN using this process is summarized in Algorithm 2.

Interpolation can be implemented in importance sampling by choosing seed points from the collection points. However, to obtain the model's predicted value, only forward propagation is needed, which is less computationally demanding. This facilitates denser sampling. Therefore, in this method, the interpolation seed points do not necessarily have to be collection points (while previous methods such as DMIS [21] necessarily required them) and can be denser in areas with substantial changes. Our OIP method dynamically adds interpolation points in areas with significant variations which facilitates a denser distribution of interpolation seeds, as illustrated in Figure 5.

Algorithm 1 Optimal Interpolation Points (N_o, N_s, β)

Input: N_o (the number of optimal interpolation points), β (the centralization parameter), N_s (the total number of sampling points).
Output: S_o (a space-time point set).

1: $S_s \leftarrow$Generate N_s points uniformly sampled from $[0, T] \times \Omega$.
2: Compute the predicted value of model by $y_i = f_\theta(x_i, t_i), (x_i, t_i) \in S_s$.
3: $\{y_i'\}_{i=1}^{N_s} \leftarrow$high-pass filtering of $\{y_i\}_{i=1}^{N_s}$ using DWT.
4: $\{s_i\}_{i=1}^{N_s} \leftarrow$linear map and histogram equalize $\{y_i'\}_{i=1}^{N_s}$.
5: Compute the sampling probability $p_i' = s_i^\beta / \sum_{i=1}^{N_s} s_i^\beta$.
6: $S_o \leftarrow N_o$ points randomly selected with probability p_i'.
7: **return** S_o.

Algorithm 2 Training of PINN via optimal interpolation points (OIP) importance sampling

Input: γ (OIP generation ratio), N_s (total number of OIP sampling points), N_r (batch size), N_c (number of collocation points), N_b (number of boundary points), N_{init} (number of initial points), t_m (iteration bound), N_t (number of interpolation seeds), η (learning rate).

Output: $\theta^{(t_m)}$ (model parameters).

Initialize $\theta^{(0)}$ (model parameters), and $k \leftarrow 0$.
$S_c \leftarrow N_c$ collocation points uniformly sampled from $[0, T] \times \Omega$.
$S_b \leftarrow N_b$ boundary points uniformly sampled from $\partial\Omega$.
$S_{init} \leftarrow N_{init}$ initial points uniformly sampled from $0 \times \Omega$.
$t_s \leftarrow t_m/4$.
$\beta \leftarrow 0$.
$N_o \leftarrow N_t \cdot \gamma, N_u \leftarrow N_t - N_o$.
while $t < t_m$ **do**
  if $t \bmod t_s = 0$ **then**
    $S_u \leftarrow N_u$ points uniformly sampled from S_c.
    $S_o \leftarrow N_o$ points sampled by Optimal Interpolation Points(N_o, N_s, β).
    $\beta \leftarrow \beta + 10$.
  end if
  Compute the loss at all points in S_u and S_o.
  Compute the approximate loss at other collocation points in S_c by interpolating method with points in S_u and S_o.
  Compute the sampling probability for all collocation points according to Equation (18).
  $S_r \leftarrow$ A batch of collocation points of size N_r randomly selected from S_c with corresponding sampling probability.
  $L_b \leftarrow \frac{1}{N_b}\sum_{i=1}^{N_b} R_b(x_i, t_i; \theta^{(k)}), (x_i, t_i) \in S_b$.
  $L_{init} \leftarrow \frac{1}{N_{init}}\sum_{i=1}^{N_{init}} R_{init}(x_i, t_i; \theta^{(k)}), (x_i, t_i) \in S_{init}$.
  $L_r \leftarrow \frac{1}{N_r}\sum_{i=1}^{N_r} \frac{1}{N_c q_i} R_r(x_i, t_i; \theta^{(k)}), (x_i, t_i) \in S_r$, where q_i is the sampling probability associated with (x_i, t_i).
  $\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_\theta (L_r + L_b + L_{init})$.
  $k \leftarrow k + 1$.
end while

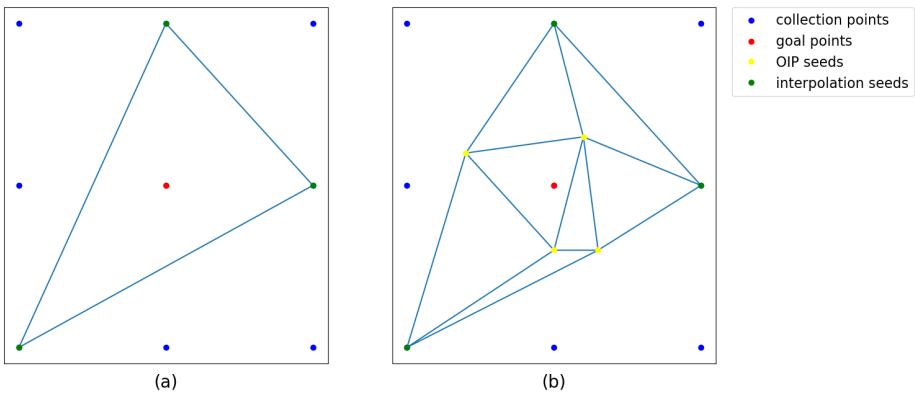

Figure 5. Comparison between the DMIS method and the OIP method we proposed. In the DMIS approach (**a**), triangular interpolation is confined to the collection points for estimating target points. In contrast, the OIP method (**b**) generates interpolation points nearer to the target points, supplementing the collection points for a more precise estimation.

4. Experiments

In this section, we use Burgers' equation [44] and the Korteweg–De Vries (KDV) equation [45] as benchmark PDE problems for our experiment. The Burgers' equation is a fundamental nonlinear partial differential equation commonly used to simulate phenom-

ena such as shock waves, turbulence, and boundary layers. It combines convection and diffusion processes, representing many practical problems involving steep gradients or discontinuities. Similarly, the KDV equation describes nonlinear wave propagation, such as solitary waves, and serves as a prototype for studying dispersive phenomena. These equations encapsulate the various challenges encountered in solving PDEs, including nonlinearity, steep gradients, and the need for adaptive resolution. We aim to verify the link between regions with significant changes in model prediction and regions with significant changes in losses. Moreover, we compare our PINN method with other PINN methods to evaluate its performance. In the experiment comparing the accuracy of model predictions, we used Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), Root Mean Squared Error (RMSE), and Maximum Error (ME) as indicators, namely,

$$\begin{cases} \mathrm{MAE}(y,\hat{y}) = \frac{1}{n}\sum_{i=1}^{n}|y_i-\hat{y}_i| \\ \mathrm{MAPE}(y,\hat{y}) = \frac{1}{n}\sum_{i=1}^{n}\frac{|y_i-\hat{y}_i|}{|y_i|} \\ \mathrm{RMSE}(y,\hat{y}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2} \\ \mathrm{ME}(y_i,\hat{y}_i) = \max(|y_1-\hat{y}_1|,|y_2-\hat{y}_2|,\ldots,|y_n-\hat{y}_n|), \end{cases} \tag{26}$$

where y_i is the actual value and $\hat{y}i$ is the model's predicted value for $i \in \{1,2,\ldots,n\}$. MAE and RMSE provide assessments of overall accuracy; MAPE evaluates relative accuracy, which is particularly important when solution magnitudes vary significantly across the domain; ME represents the worst-case scenario, emphasizing model robustness. All training and tests are conducted on an NVIDIA GeForce RTX 3060 GPU using Pytorch [46].

The Burgers' Equation is a PDE that models the propagation and reflection of shock waves. It is widely used in areas such as traffic flow, fluid mechanics, and aerodynamics. For the low-viscosity Burgers equation, steep gradients and discontinuities (shocks) commonly occur. PINN's accuracy suffers in these regions due to insufficient sampling density in rapidly changing areas and the inability of gradient-based optimization to effectively propagate errors across the domain. Additionally, the introduction of nonlinear advection makes the solution highly sensitive to initial and boundary conditions, increasing the difficulty for PINN to obtain approximate solutions across the entire domain. A typical solution of the Burgers equation is illustrated in Figure 6a. In this paper, we consider the one-dimensional Burgers' Equation in the spatial domain $\Omega = [-1,1]$:

$$\begin{cases} \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \frac{0.04}{\pi}\frac{\partial^2 u}{\partial x^2}, & (x,t) \in [-1,1]\times[0,1] \\ u(x,0) = -\sin(\pi x), & x \in [-1,1] \\ u(-1,t) = u(1,t) = 0, & t \in [0,1] \end{cases} \tag{27}$$

The KDV equation is a partial differential equation that describes the phenomenon of solitons, frequently found in various branches of physics like nonlinear optics, plasma theory, and soliton field theory. The interaction between nonlinear advection and dispersion in the KDV equation leads to complex multi-scale phenomena, requiring PINN to resolve small, sharp structures with high resolution. Furthermore, KDV equations typically involve periodic boundary conditions, presenting challenges for accurate constraint enforcement during training. The equation's third-order derivatives make it suitable for evaluating PINN's efficiency in solving high-order derivatives. A typical solution of the KDV equation is shown in Figure 6b. In this paper, we consider the one-dimensional KDV equation in the spatial domain $\Omega = [-1,1]$:

$$\begin{cases} \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + 0.0025\frac{\partial^3 u}{\partial x^3} = 0, & (x,t) \in [-1,1] \times [0,1] \\ u(x,0) = \cos(\pi x), & x \in [-1,1] \\ u(-1,t) = u(1,t), & t \in [0,1] \\ \frac{\partial u}{\partial x}(-1,t) = \frac{\partial u}{\partial x}(1,t), & t \in [0,1] \end{cases} \tag{28}$$

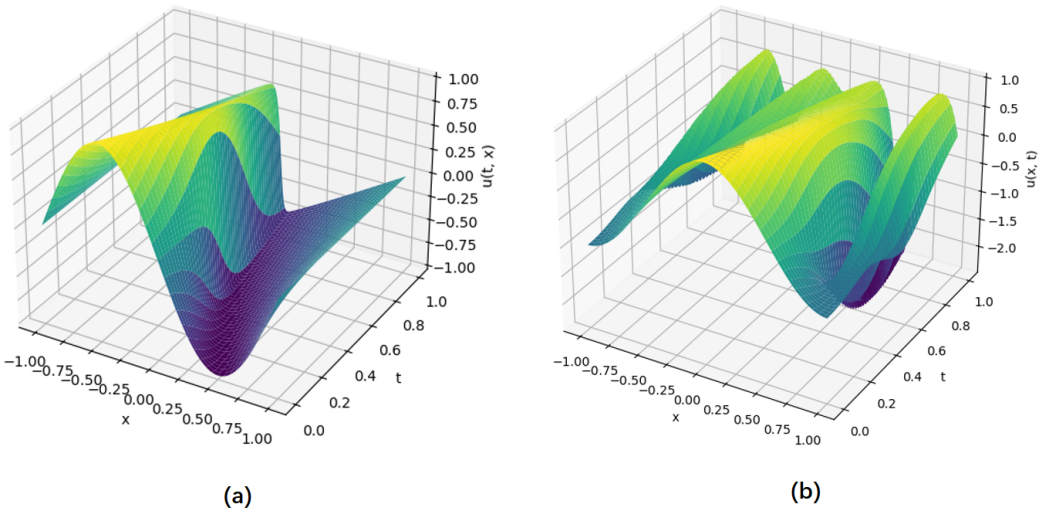

Figure 6. Typical solutions for the Burgers and KDV equations, (**a**) Burgers' equation, (**b**) KDV equation.

We apply uniform sampling to the PINN model over multiple rounds. This method allows us to investigate the relationship between regions with significant loss changes and those with significant changes in the model's predicted values.

To measure the similarity between two sets of points, we employ the following formula:

$$\Phi(A,B) = 1 - \frac{\sum_{a \in A} \sum_{b \in B} d(a,b)}{|A| \cdot |B|}, \tag{29}$$

where, A and B represent sets of points, $\Phi(A,B)$ denotes the similarity between A and B, $|\cdot|$ denotes the number of elements in the set, and $d(a,b)$ represents the Euclidean distance between points a and b.

The numerical solution computed by Py-PDE [41] serves as the ground truth. To examine the model's generalization ability, the dataset is divided based on its time domain. Given that PINN already demonstrates high interpolation accuracy while extrapolation accuracy still needs improvement [47], our experiments primarily test whether the proposed method exhibits good extrapolation capabilities. Therefore, for the time domain $[0,T]$, we use the data from $[0,0.6T]$ for training, $[0.6T,0.8T]$ for validation, and $[0.8T,T]$ for testing. This setup trains the model on historical data to predict future states of the physical system, simulating real-world scenarios. Such division challenges the model's generalization ability, highlighting its capability to handle temporal extrapolation.

We explored several variants of PINN, including the original PINN [13] (denoted as PINN-O), PINN based on importance sampling [20] (denoted as PINN-I), and PINN based on dynamic grid importance sampling [21] (denoted as PINN-D). We also tested our proposed PINN using the best interpolation point importance sampling method. Other interpolation methods were sourced from triangle interpolation [48,49] (denoted as PINN-OIP-tri) and nearest neighbor interpolation [50] (denoted as PINN-OIP-ne). These variants are summarized in Table 1.

Table 1. The summary of variants of PINN for our experiment. (IS: Importance Sampling; DS: Dynamic Seeds).

Model	IS	DS	Interpolation Method	Seed Sampling Method
PINN-O	No	-	-	-
PINN-I	Yes	No	Nearest Neighbor Interpolation	Uniform Sampling
PINN-D	Yes	Yes	Triangle Interpolation	Dynamic Mesh
PINN-OIP-ne	Yes	Yes	Nearest Neighbor Interpolation	OIP(Our Method)
PINN-OIP-tri	Yes	Yes	Triangle Interpolation	OIP(Our Method)

The network structure of PINN and the various hyperparameters are outlined in Table 2. In Table 2, "S" stands for the number of interpolation seed points, while "β" represents the ratio of the best interpolation points. "Max steps" refers to the number of training iterations. "N_r", "N_{init}" and "N_b" are the batch sizes for PDE residuals, initial conditions, and boundary conditions data, respectively. All training was conducted using the Adam optimizer.

Table 2. The network structure and hyper-parameters in the PINN for the Burgers' equation and the Korteweg–De Vries equation.

Equation	Network		Hyper-Parameters				Batchsize		
	Depth	Width	Learning Rate	S	β	Max Steps	N_r	N_{init}	N_b
Burgers	3	32	0.005	1000	0.5	20,000	10,000	500	500
KDV	4	64	0.001	1000	0.5	50,000	50,000	50	50

Table 3 presents the results of the correlation experiment, which includes uniformly selected points, points with significant loss changes, and points with high predicted value changes. In Table 3, N denotes the percentage of the number of sampling points compared to the total.

Table 3. Similarity between the set of points acquired by high-frequency filtering via wavelet transform sampling or uniform sampling, and the set of points where significant changes in loss occur.

Equation	Burgers'Equation			KDV Equation		
N	10%	20%	30%	10%	20%	30%
Uniform Sampling	0.251	0.244	0.235	0.125	0.155	0.185
Most Dramatic Predicted Value Change Sampling	0.539	0.501	0.471	0.656	0.548	0.525

Figure 7 displays the distribution of points that reflect significant changes in both predicted values and losses. There is a noticeable correlation between them. The experiments demonstrate the correlation across different equations, indicating that steep regions in model values lead to training difficulties and higher loss values in these areas. This provides justification for using DWT to extract relevant regions based on model predictions and increasing seed points in these areas. Figure 8 displays the distribution of sampling points with various β values. These values were obtained using the OIP method on the Burgers equation model, highlighting the region where the loss undergoes significant changes as β increases.

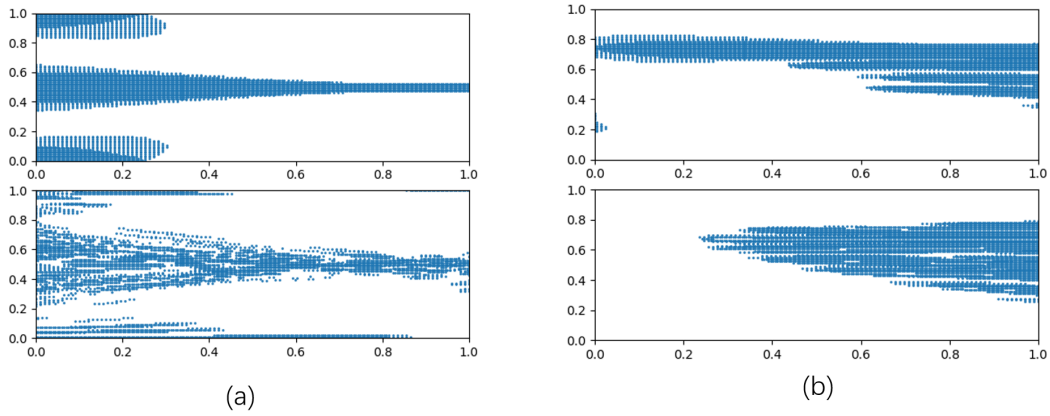

Figure 7. Distribution of the top 10% points with the most significant predicted value changes (obtained using DWT). (**a**) the Burgers equation (**b**) the Korteweg–De Vries equation.

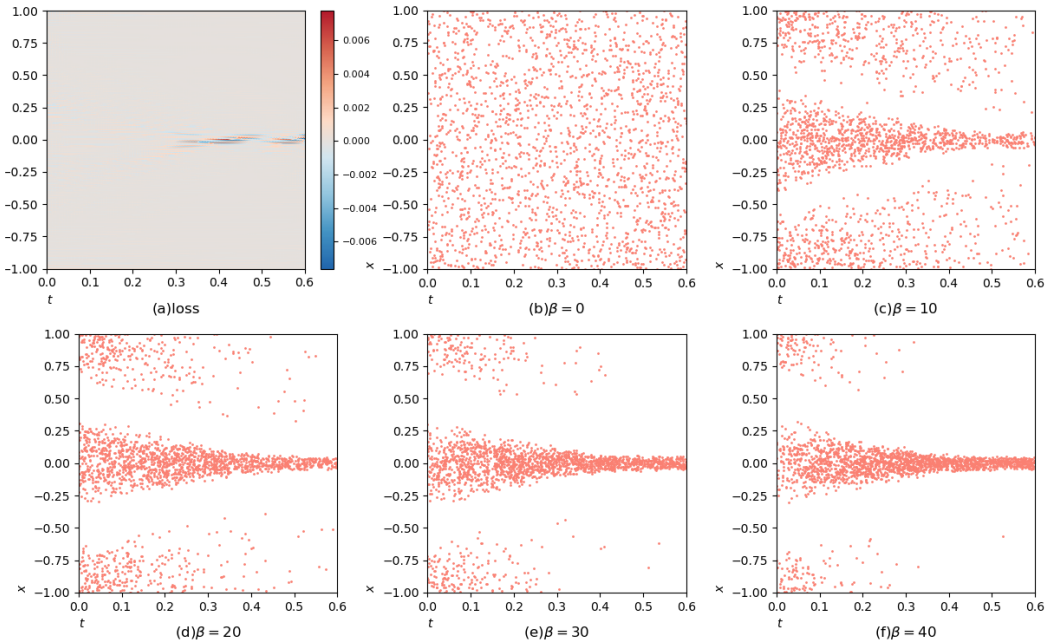

Figure 8. Distribution of sampling points in the PINN for Burgers equation, (**a**) represents the model's loss, while (**b**–**f**) demonstrate the distribution of interpolation seed points. Here, (**b**) shows the case when $\beta = 0$, which is equivalent to uniform sampling.

Tables 4 and 5 display the predictions of the PINN-O, PINN-I, PINN-D, PINN-OIP-NE, and PINN-OIP-TRI for the Burgers' equation and the KDV equation, respectively. A lower ME value suggests minimal deviation during prediction, meaning our model maintains stable performance in most scenarios. Moreover, a lower MAE value represents a smaller difference between predicted and actual values, indicating better accuracy. MAPE, which considers error relative to actual values, provides insight into prediction performance across different data orders. Lower MAPE values show improved accuracy across various data points. RMSE, being sensitive to larger errors, also shows lower values with our method, indicating consistently low error levels. Table 6 shows the comparison of model training times. Compared to the original PINN model and other importance sampling-based PINN models, our proposed method requires additional computation time due to the DWT-based seed point selection process. However, this additional time is relatively small compared to the total training time, increasing by only 3.9% and 9.7% for the two equations, respectively. The PINN trained with the OIP method we proposed outperforms others in almost all the indicators mentioned previously.

Table 4. Comparison of PINN-O, PINN-I, PINN-D, PINN-OIP-NE, PINN-OIP-TRI for the Burgers' Equation.

Method	ME	MAE	RMSE	MAPE
PINN-O	*0.451*	*0.0805*	*0.1158*	**0.4293**
PINN-I	0.437	0.0199	0.0579	0.0658
PINN-D	0.386	0.0185	0.0478	0.0622
PINN-OIP-NE (ours)	0.332	0.0167	**0.0378**	*0.0997*
PINN-OIP-TRI (ours)	**0.323**	**0.0150**	0.0393	0.0532

Italic indicates the worst performance, and **bold** indicates the best performance.

Table 5. Comparison of PINN-O, PINN-I, PINN-D, PINN-OIP-NE, PINN-OIP-TRI for the Korteweg–De Vries Equation.

Method	ME	MAE	RMSE	MAPE
PINN-O	*2.138*	0.395	*0.557*	**1.497**
PINN-I	1.354	*0.421*	0.550	*2.231*
PINN-D	1.878	0.326	0.492	1.943
PINN-OIP-NE (ours)	1.106	0.351	0.463	1.682
PINN-OIP-TRI (ours)	**0.922**	**0.212**	**0.280**	1.560

Italic indicates the worst performance, and **bold** indicates the best performance.

Table 6. Comparison of PINN-O, PINN-D, PINN-OIP-tri in training time.

Equation	Training Time (s)		
	PINN-O	PINN-D	PINN-OIP-tri
Burgers	299	436	453
KDV	789	913	1002

5. Conclusions

In this paper, we presented a novel method that applies importance sampling to the training of Physics Informed Neural Networks (PINN). In this method, we employed the predicted values from the PINN to optimally estimate the sampling probability of the interpolation seed points. Specifically, we use DWT to identify regions with the most dramatic changes in the solution domain and increase seed points in these areas, thereby obtaining more accurate interpolation estimates to enhance the effectiveness of importance sampling. Experiments demonstrate that our method effectively improves the accuracy of importance sampling-based PINN models. Compared to current state-of-the-art methods, it reduces the root mean square error by 20.9% for the Burgers equation and 43% for the KDV equation, while only increasing training time by 3.9% and 9.7%, respectively.

Several limitations in our research warrant discussion. While our proposed method introduces additional sampling processes using DWT that do not significantly impact computational cost in current experiments, these costs may increase as problem scales expand or required resolution increases. When extending to high-dimensional problems, the computational burden of DWT for extracting high-frequency regions may grow, necessitating more effective strategies to balance computational load and accuracy. Additionally, the method's hyperparameters may require adjustment for different problems to achieve optimal results. In future research, we plan to explore applying the proposed method to higher-dimensional problems, investigate more efficient interpolation computation methods to further improve computational efficiency, and examine the feasibility of applying adaptive sampling and rendering techniques to real-time simulation.

Author Contributions: Methodology was conducted by Y.Z. and H.L.; Formal analysis was conducted by Z.W. (Zhaoxiong Wu), Z.W. (Zhe Wang) and T.W. and the final revision and English polishing were conducted by H.L. and T.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Key R&D Program of China grant number 2021YFB3802200.

Data Availability Statement: Our source code is available at https://github.com/plantashe/OIP (accessed on 27 November 2024).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Lambe, C.G.; Tranter, C.J. *Differential Equations for Engineers and Scientists*; Courier Dover Publications: Mineola, NY, USA, 2018.
2. Zhang, X.; Wang, L.; Helwig, J.; Luo, Y.; Fu, C.; Xie, Y.; Liu, M.; Lin, Y.; Xu, Z.; Yan, K.; et al. Artificial intelligence for science in quantum, atomistic, and continuum systems. *arXiv* **2023**, arXiv:2307.08423.
3. Bonnet, F.; Mazari, J.; Cinnella, P.; Gallinari, P. Airfrans: High fidelity computational fluid dynamics dataset for approximating reynolds-averaged navier–stokes solutions. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 23463–23478.
4. Brandstetter, J.; Berg, R.V.D.; Welling, M.; Gupta, J.K. Clifford neural layers for pde modeling. *arXiv* **2022**, arXiv:2209.04934.
5. Zobeiry, N.; Humfeld, K.D. A physics-informed machine learning approach for solving heat transfer equation in advanced manufacturing and engineering applications. *Eng. Appl. Artif. Intell.* **2021**, *101*, 104232. [CrossRef]
6. Ames, W.F. *Numerical Methods for Partial Differential Equations*; Academic Press: Cambridge, MA, USA, 2014.
7. Quarteroni, A.; Valli, A. *Numerical Approximation of Partial Differential Equations*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2008; Volume 23.
8. Bartels, S. *Numerical Approximation of Partial Differential Equations*; Springer: Berlin/Heidelberg, Germany, 2016; Volume 64.
9. Wang, H.; Fu, T.; Du, Y.; Gao, W.; Huang, K.; Liu, Z.; Chandak, P.; Liu, S.; Van Katwyk, P.; Deac, A.; et al. Scientific discovery in the age of artificial intelligence. *Nature* **2023**, *620*, 47–60. [CrossRef] [PubMed]
10. Qiao, Z.; Christensen, A.S.; Welborn, M.; Manby, F.R.; Anandkumar, A.; Miller, T.F., III. Informing geometric deep learning with electronic interactions to accelerate quantum chemistry. *Proc. Natl. Acad. Sci. USA* **2022**, *119*, e2205221119. [CrossRef] [PubMed]
11. Townshend, R.J.; Eismann, S.; Watkins, A.M.; Rangan, R.; Karelina, M.; Das, R.; Dror, R.O. Geometric deep learning of RNA structure. *Science* **2021**, *373*, 1047–1051. [CrossRef]
12. Karpatne, A.; Watkins, W.; Read, J.; Kumar, V. Physics-guided neural networks (pgnn): An application in lake temperature modeling. *arXiv* **2017**, arXiv:1710.11431.
13. Raissi, M.; Perdikaris, P.; Karniadakis, G.E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **2019**, *378*, 686–707. [CrossRef]
14. Faroughi, S.A.; Pawar, N.M.; Fernandes, C.; Raissi, M.; Das, S.; Kalantari, N.K.; Kourosh Mahjour, S. Physics-guided, physics-informed, and physics-encoded neural networks and operators in scientific computing: Fluid and solid mechanics. *J. Comput. Inf. Sci. Eng.* **2024**, *24*, 040802. [CrossRef]
15. Li, Z.; Kovachki, N.; Azizzadenesheli, K.; Liu, B.; Bhattacharya, K.; Stuart, A.; Anandkumar, A. Fourier neural operator for parametric partial differential equations. *arXiv* **2020**, arXiv:2010.08895.
16. Cuomo, S.; Di Cola, V.S.; Giampaolo, F.; Rozza, G.; Raissi, M.; Piccialli, F. Scientific machine learning through physics–informed neural networks: Where we are and what's next. *J. Sci. Comput.* **2022**, *92*, 88. [CrossRef]
17. Gao, H.; Sun, L.; Wang, J.X. PhyGeoNet: Physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state PDEs on irregular domain. *J. Comput. Phys.* **2021**, *428*, 110079. [CrossRef]
18. Viana, F.A.; Nascimento, R.G.; Dourado, A.; Yucesan, Y.A. Estimating model inadequacy in ordinary differential equations with physics-informed neural networks. *Comput. Struct.* **2021**, *245*, 106458. [CrossRef]
19. Yang, L.; Zhang, D.; Karniadakis, G.E. Physics-informed generative adversarial networks for stochastic differential equations. *SIAM J. Sci. Comput.* **2020**, *42*, A292–A317. [CrossRef]
20. Nabian, M.A.; Gladstone, R.J.; Meidani, H. Efficient training of physics-informed neural networks via importance sampling. *Comput.-Aided Civ. Infrastruct. Eng.* **2021**, *36*, 962–977. [CrossRef]

21. Yang, Z.; Qiu, Z.; Fu, D. Dmis: Dynamic mesh-based importance sampling for training physics-informed neural networks. In Proceedings of the AAAI Conference on Artificial Intelligence, Washington, DC, USA, 7–14 February 2023; Volume 37, pp. 5375–5383.
22. Jagtap, A.D.; Karniadakis, G.E. Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. *Commun. Comput. Phys.* **2020**, *28*, 2002–2041. [CrossRef]
23. Wang, S.; Yu, X.; Perdikaris, P. When and why PINNs fail to train: A neural tangent kernel perspective. *J. Comput. Phys.* **2022**, *449*, 110768. [CrossRef]
24. Daw, A.; Bu, J.; Wang, S.; Perdikaris, P.; Karpatne, A. Mitigating propagation failures in physics-informed neural networks using retain-resample-release (r3) sampling. *arXiv* **2022**, arXiv:2207.02338.
25. Wu, C.; Zhu, M.; Tan, Q.; Kartha, Y.; Lu, L. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Comput. Methods Appl. Mech. Eng.* **2023**, *403*, 115671. [CrossRef]
26. Mao, Z.; Meng, X. Physics-informed neural networks with residual/gradient-based adaptive sampling methods for solving partial differential equations with sharp solutions. *Appl. Math. Mech.* **2023**, *44*, 1069–1084. [CrossRef]
27. Amari, S.I. Backpropagation and stochastic gradient descent method. *Neurocomputing* **1993**, *5*, 185–196. [CrossRef]
28. Margossian, C.C. A review of automatic differentiation and its efficient implementation. *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.* **2019**, *9*, e1305. [CrossRef]
29. Tokdar, S.T.; Kass, R.E. Importance sampling: A review. *Wiley Interdiscip. Rev. Comput. Stat.* **2010**, *2*, 54–60. [CrossRef]
30. Evans, M.; Swartz, T. *Approximating Integrals via Monte Carlo and Deterministic Methods*; OUP: Oxford, UK, 2000; Volume 20.
31. Müller, T.; McWilliams, B.; Rousselle, F.; Gross, M.; Novák, J. Neural importance sampling. *ACM Trans. Graph. (ToG)* **2019**, *38*, 1–19. [CrossRef]
32. Bugallo, M.F.; Elvira, V.; Martino, L.; Luengo, D.; Miguez, J.; Djuric, P.M. Adaptive Importance Sampling: The past, the present, and the future. *IEEE Signal Process. Mag.* **2017**, *34*, 60–79. [CrossRef]
33. Li, M.; Zhang, T.; Chen, Y.; Smola, A.J. Efficient mini-batch training for stochastic optimization. In Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, New York, NY, USA, 24–27 August 2014; pp. 661–670.
34. Csiba, D.; Richtárik, P. Importance sampling for minibatches. *J. Mach. Learn. Res.* **2018**, *19*, 1–21.
35. Zhao, P.; Zhang, T. Stochastic optimization with importance sampling for regularized loss minimization. In Proceedings of the International Conference on Machine Learning. PMLR, Lille, France, 7–9 July 2015; pp. 1–9.
36. Katharopoulos, A.; Fleuret, F. Biased importance sampling for deep neural network training. *arXiv* **2017**, arXiv:1706.00043.
37. Burrus, C.S.; Gopinath, R.A.; Guo, H. *Wavelets and Wavelet Transforms*; Rice University: Houston, TX, USA, 1998; Volume 98.
38. Zhang, D.; Zhang, D. Wavelet transform. In *Fundamentals of Image Data Mining: Analysis, Features, Classification and Retrieval*; Springer: Berlin/Heidelberg, Germany, 2019; pp. 35–44.
39. Zeebaree, D.Q.; Haron, H.; Abdulazeez, A.M.; Zebari, D.A. Machine learning and region growing for breast cancer segmentation. In Proceedings of the 2019 International Conference on Advanced Science and Engineering (ICOASE), Zakho/Duhok, Iraq, 2–4 April 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 88–93.
40. Brifcani, A.M.A.; Al-Bamerny, J.N. Image compression analysis using multistage vector quantization based on discrete wavelet transform. In Proceedings of the 2010 International Conference on Methods and Models in Computer Science (ICM2CS-2010), New Delhi, India, 13–14 December 2010; IEEE: Piscataway, NJ, USA, 2010; pp. 46–53.
41. Zwicker, D. py-pde: A python package for solving partial differential equations. *J. Open Source Softw.* **2020**, *5*, 2158. [CrossRef]
42. Othman, G.; Zeebaree, D.Q. The applications of discrete wavelet transform in image processing: A review. *J. Soft Comput. Data Min.* **2020**, *1*, 31–43.
43. SM, P. Adaptive histogram equalization and its variations. *Comput. Graph. Image Process.* **1977**, *6*, 184–195.
44. Bonkile, M.P.; Awasthi, A.; Lakshmi, C.; Mukundan, V.; Aswin, V. A systematic literature review of Burgers' equation with recent advances. *Pramana* **2018**, *90*, 69. [CrossRef]
45. Zhang, D.J.; Zhao, S.L.; Sun, Y.Y.; Zhou, J. Solutions to the modified Korteweg–de Vries equation. *Rev. Math. Phys.* **2014**, *26*, 1430006. [CrossRef]
46. Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; et al. Pytorch: An imperative style, high-performance deep learning library. *Adv. Neural Inf. Process. Syst.* **2019**, *32*, 8026–8037.
47. Kim, J.; Lee, K.; Lee, D.; Jhin, S.Y.; Park, N. DPM: A novel training method for physics-informed neural networks in extrapolation. In Proceedings of the AAAI Conference on Artificial Intelligence, Virtual, 2–9 February 2021; Volume 35, pp. 8146–8154.
48. Watson, D.; Philip, G. Triangle based interpolation. *J. Int. Assoc. Math. Geol.* **1984**, *16*, 779–795. [CrossRef]

49. Lee, D.T.; Schachter, B.J. Two algorithms for constructing a Delaunay triangulation. *Int. J. Comput. Inf. Sci.* **1980**, *9*, 219–242. [CrossRef]
50. Rukundo, O.; Cao, H. Nearest neighbor value interpolation. *arXiv* **2012**, arXiv:1211.1768.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.